University of Yahia Fares Medea

Faculty of Sciences & Technology

Department of Computer Science

Academic Year 2025 / 2026

Go · Python · FastAPI · Whisper · HF Spaces

Tahkik Inference Server

Technical Report

Server Architecture, API & Deployment

Benhadjer Mohamed

Group 2

L3 Computer Science

Bouziani Alaa Eddine

Group 2

L3 Computer Science

Contents

1. Introduction

This technical report documents the server-side infrastructure of the Tahkik project — an AI-powered Quranic recitation analysis system specializing in the Warsh 'an Nafi' riwaya. The server bridges the Flutter mobile application with the Whisper-based inference model, handling audio uploads, forwarding them to the machine learning backend, and returning structured transcription results.

The system is composed of two primary components: a Go HTTP API gateway that receives client requests and a Python inference backend (deployed as a Hugging Face Space) that runs the fine-tuned Whisper model. This report covers the architecture, component details, data pipeline, API contracts, deployment procedures, and troubleshooting.

2. System Architecture

2.1 System Overview

The Tahkik server follows a proxy architecture where the Go API server acts as a gateway between the mobile client and the ML inference backend. This design keeps authentication tokens server-side and allows swapping inference backends without modifying the Flutter codebase.

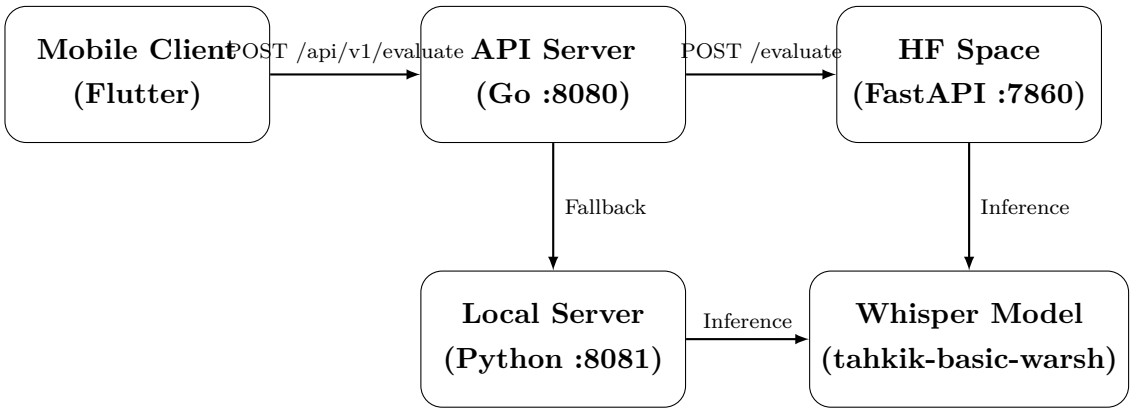

Figure 2.1: High-level system architecture

2.2 Design Justification

The architectural decisions were driven by the following constraints:

- **Proxy Pattern:** The Go server acts as a reverse proxy, shielding the inference backend from direct client access. This enables server-side token management and backend flexibility.
- **Go for the API Gateway:** Go's standard library `net/http` provides a production-ready HTTP server with minimal dependencies, fast compilation, and low memory footprint.
- **faster-whisper (CTranslate2):** The production deployment uses `faster-whisper` with INT8 quantization, achieving approximately 4× faster inference compared to the PyTorch-based backend used during development.

- **Hugging Face Spaces:** Chosen for zero-infrastructure deployment with Docker SDK support, automatic scaling, and optional GPU hardware (T4/L4/A100).
- **WebSocket Streaming:** The `/ws/stream` endpoint enables real-time partial transcription, providing immediate user feedback during recitation without waiting for the full recording to complete.

3. Component Details

3.1 Go API Server

3.1.1 Overview

The Go API server is the entry point for all client requests. It listens on port 8080 (configurable via `PORT` environment variable) and exposes two endpoints: `POST /api/v1/evaluate` for audio evaluation and `GET /health` for liveness checks.

3.1.2 Technology

Technology Stack

- **Language**: Go 1.26.2
- **Framework**: Standard library (`net/http`)
- **Key Packages**: `encoding/json`, `mime/multipart`, `io`

3.1.3 Key Implementation Details

The server uses Go's built-in `http.NewServeMux` for routing. The evaluate handler validates the uploaded audio file (max 50 MB, supported formats: `.wav`, `.mp3`, `.m4a`, `.flac`, `.ogg`), reads the binary data, and delegates to the ML inference client.

Listing 3.1: API entry point (cmd/api/main.go)

```
1  func main() {
2      port := os.Getenv("PORT")
3      if port == "" { port = "8080" }
4
5      mux := http.NewServeMux()
6      mux.HandleFunc("/api/v1/evaluate", handlers.Evaluate)
7      mux.HandleFunc("/health", func(w http.ResponseWriter, r *http
           .Request) {
8          w.Header().Set("Content-Type", "application/json")
9          w.Write([]byte(`{"status":"ok"}`))
10     })
11
```

```
    log.Printf("Tahkik server listening on :%s", port)
    http.ListenAndServe(":"+port, mux)
}
```

3.1.4 Inference Client

The `internal/ml/inference.go` module forwards audio to the Python backend as `multipart/form-da`
It supports both local and remote backends via the `INFERENCE_SERVER_URL` environment variable.

Listing 3.2: Inference result structure

```
type Result struct {
    Transcription    string  `json:"transcription"`
    ConfidenceScore  float64 `json:"confidence_score"`
    ProcessingTimeMs int     `json:"processing_time_ms"`
}
```

Key characteristics:

- HTTP client timeout: 5 minutes (accommodates large audio files)
- Sends `Authorization: Bearer {HF_TOKEN}` header when available
- Parses error responses from both `detail` and `error` JSON fields

3.2 Python Inference Server (Local Development)

3.2.1 Overview

The local Python inference server loads the Whisper model once at startup and serves HTTP requests on port 8081. It is designed for development and testing without internet access.

3.2.2 Technology

Technology Stack

- **Language**: Python 3.10+
- **Framework**: `http.server` (standard library)
- **Key Libraries**: `torch`, `transformers`, `librosa`, `soundfile`

3.2.3 Model Loading

The server prints `[inference] READY` when the model is fully loaded, which the Go server uses as a startup signal. If the fine-tuned checkpoint is missing `lang_to_id` in its generation config, these fields are patched from `openai/whisper-base`.

Listing 3.3: Model loading with generation config patching

```
TAHKIK_MODEL = "benhadjermed/tahkik-small-warsh"

processor = WhisperProcessor.from_pretrained(
    "openai/whisper-base", language="Arabic", task="transcribe"
)
model = WhisperForConditionalGeneration.from_pretrained(
    TAHKIK_MODEL, torch_dtype=torch_dtype
).to(device)

# Patch missing generation config from base model
if model.generation_config.lang_to_id is None:
    _base = WhisperForConditionalGeneration.from_pretrained(
        "openai/whisper-base"
    )
    model.generation_config.lang_to_id = _base.generation_config.
       lang_to_id
```

3.3 HF Space FastAPI Server (Production)

3.3.1 Overview

The production inference server uses **faster-whisper** (CTranslate2 backend) with INT8 quantization for significantly faster inference. It is deployed as a Docker-based Hugging Face Space.

3.3.2 Technology

Technology Stack

- **Language**: Python 3.10
- **Framework**: FastAPI + Uvicorn
- **ML Runtime**: faster-whisper (CTranslate2, INT8)
- **Key Libraries**: `numpy`, `librosa`, `python-multipart`
- **Deployment**: Docker on Hugging Face Spaces

3.3.3 CTranslate2 Model Conversion

The fine-tuned Whisper model is converted from PyTorch format to CTranslate2 INT8 for production:

Listing 3.4: Model conversion to CTranslate2

```
from ctranslate2.converters import TransformersConverter

converter = TransformersConverter(
    model_name_or_path=local_hf, low_cpu_mem_usage=True
)
converter.convert(ct2_dir, quantization="int8", force=True)
```

3.3.4 Hallucination Filtering

Whisper models trained on YouTube data produce hallucination phrases on silence. The server filters these known phrases:

Known Hallucination Phrases

- Arabic phrases: “Shukran lak”, “Shukran lakum”, “Tarjamat Nancy Qanqar”, “Tabi’una” (12 variants total)
- English: `Thank you`, `thank you`
- Punctuation-only: `.`

3.3.5 WebSocket Streaming Protocol

The `/ws/stream` endpoint enables real-time transcription using the following protocol:

Table 3.1: Client → Server messages

Frame	Type	Description
Binary	PCM audio	Raw 16-bit signed LE, 16 kHz, mono
Text	`{"type":"stop"}`	End of recording
Text	`{"type":"ping"}`	Heartbeat keepalive
Text	`{"type":"context","text":"..."}`	Ayah text for decoding bias
Text	`{"type":"reset"}`	Reset session state

Table 3.2: Server → Client messages

Type	Description
`partial`	Intermediate transcription (`text` field)
`final`	Final result (`text`, `confidence`, `processing_time_ms`)
`error`	Error message (`message` field)
`pong`	Heartbeat response

Streaming Implementation Highlights

- **Sliding window**: Transcribes the last 8 seconds as one contiguous window, avoiding word-cutting at chunk boundaries
- **Global inference lock**: One inference at a time to prevent resource contention
- **Generation counter**: Stale background tasks discard their output after session resets
- **OOM protection**: Drops audio buffer after 10 seconds of pure silence
- **Final refinement**: On `stop`, sends immediate result then refines in background using full buffer (capped at 30s)

4. Data Pipeline

4.1 Audio Processing Flow

When an audio file arrives at the server, it passes through the following stages:

Phase 1 — Audio Reception

The Go server receives the multipart upload, validates the file extension and size (max 50 MB), reads the binary data into memory, and forwards it to the inference backend.

Phase 2 — Audio Preprocessing

The Python backend resamples the audio to 16 kHz using `librosa.load()`, then splits it into overlapping chunks of 30 seconds with 1-second overlap to fit Whisper's context window.

Phase 3 — Inference

Each chunk is passed through the Whisper model for transcription. The model generates Arabic text with confidence scores derived from token-level softmax probabilities.

Phase 4 — Result Assembly

Chunk transcriptions are concatenated. Confidence scores are averaged across all chunks. The final JSON result is returned to the Go server and forwarded to the client.

4.2 Audio Chunking Algorithm

Listing 4.1: Audio splitting with overlap

```
CHUNK_LENGTH_S = 30
OVERLAP_S = 1
SAMPLE_RATE = 16000

def split_audio(audio_array, sr=SAMPLE_RATE):
    chunk_len = int(CHUNK_LENGTH_S * sr)
    step_len = int((CHUNK_LENGTH_S - OVERLAP_S) * sr)
    chunks = []
```

```
    start = 0
    while start < len(audio_array):
        end = min(start + chunk_len, len(audio_array))
        chunks.append(audio_array[start:end])
        start += step_len
        remaining = len(audio_array) - start
        if 0 < remaining < 2 * sr:
            chunks[-1] = audio_array[start - step_len:]
            break
    return chunks
```

4.3 Confidence Scoring

Confidence Calculation Methods

- **Local server (PyTorch)**: Mean of max token-level softmax probabilities across all generated tokens: $\text{conf} = \frac{1}{T}\sum_{t=1}^{T} \max_v P(v|t)$
- **HF Space (faster-whisper)**: Converts `avg_logprob` per segment to confidence via $\text{conf} = e^{\max(\text{avg\_logprob}, -5.0)}$

5. Setup and Deployment

5.1 Option 1: Remote HF Space (Recommended)

Setup Commands

```
export INFERENCE_SERVER_URL=https://benhadjermed-tahkik-inference.hf.space
export HF_TOKEN=hf_xxxxxxxxxx
export PORT=8080
go run ./cmd/api
```

5.2 Option 2: Local Python Server (Development)

Step 1 — Python Environment

```
cd python
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

Step 2 — Start Inference Server

```
python inference_server.py -port 8081
```

Wait for: `[inference] READY`

Step 3 — Start Go Server

```
export PORT=8080
go run ./cmd/api
```

5.3 HF Space Deployment

Step 1 — Prepare Space

Create a new Space on huggingface.co: Dashboard → New Space → SDK: Docker → name: `tahkik-inference`

Step 2 — Deploy

```
git clone https://huggingface.co/spaces/benhadjermed/tahkik-inference
cp python/hf_space/* .
git add .  && git commit -m "initial inference server" && git push
```

5.3.1 Dockerfile

Listing 5.1: HF Space Dockerfile

```
FROM python:3.10-slim
RUN useradd -m -u 1000 user
WORKDIR /home/user/app
COPY --chown=user requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY --chown=user . .
ENV HF_HOME=/tmp/huggingface_cache
USER user
EXPOSE 7860
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port",
   "7860"]
```

5.4 Environment Variables

Table 5.1: Configuration environment variables

Variable	Default	Purpose
`INFERENCE_SERVER_URL`	`http://localhost:8081`	Backend inference server URL
`HF_TOKEN`	None	Bearer token for HF access
`PORT`	`8080`	Go server listen port

5.5 Hardware Recommendations

Table 5.2: Hardware tiers for HF Space deployment

Tier	Hardware	Cold Start	Latency/Request
Free	CPU	∼60 s	∼3–5 s
Production	T4 GPU ($0.40/hr)	faster	∼0.5–1 s

6. API Reference

6.1 Go Server Endpoints (Client-Facing)

6.1.1 POST /api/v1/evaluate

Upload an audio file for Arabic Quranic recitation transcription.

Request Format

Content-Type: `multipart/form-data`

Field: `audio` (file)

Supported formats: `.wav`, `.mp3`, `.m4a`, `.flac`, `.ogg`

Max size: 50 MB

Success Response (200 OK)

```
{
  "status": "success",
  "data": {
    "transcription": "<Arabic transcription text>",
    "confidence_score": 0.9423,
    "processing_time_ms": 1350
  }
}
```

Error Response (4xx/5xx)

```
{
  "status": "error",
  "message": "unsupported audio format: .xyz"
}
```

6.1.2 GET /health

Health Check Response (200 OK)

```
{"status": "ok"}
```

6.2 Response Fields

Table 6.1: API response data fields

Field	Type	Description
`data.transcription`	string	Arabic transcription of the recitation
`data.confidence_score`	float	Average token-level confidence (0–1)
`data.processing_time_ms`	int	Time in model (ms), excluding network

6.3 Flutter Integration Example

Listing 6.1: Dart/Flutter API call example

```dart
final dio = Dio(BaseOptions(baseUrl: 'http://<go-server>:8080'));

Future<Map<String, dynamic>> evaluate(File audioFile) async {
  final formData = FormData.fromMap({
    'audio': await MultipartFile.fromFile(
      audioFile.path,
      filename: path.basename(audioFile.path),
    ),
  });
  final response = await dio.post('/api/v1/evaluate', data:
     formData);
  return response.data as Map<String, dynamic>;
}
```

7. Troubleshooting

7.0.1 Inference Server Error 404

Diagnosis

The Go server cannot reach the inference backend. The `INFERENCE_SERVER_URL` may be incorrect or have a trailing slash.

Resolution

Ensure the URL is correct with no trailing slash:

```
export INFERENCE_SERVER_URL=https://benhadjermed-tahkik-inference.hf.space
```

7.0.2 Model Download Timeout

Diagnosis

First-time model download from HuggingFace can take several minutes, especially for the full Whisper weights. The local server prints `[inference] downloading/loading model...` during this process.

Resolution

Wait for the download to complete. Ensure a stable internet connection. Subsequent starts will use cached weights.

7.0.3 HF Token Required

Diagnosis

The model repository or HF Space is set to private, requiring authentication.

Resolution

Set the `HF_TOKEN` environment variable:

```
export HF_TOKEN=hf_xxxxxxxxxx
```

8. Performance Metrics

8.1 Inference Latency Analysis

The inference latency of the Tahkik server varies significantly depending on the hardware configuration and the length of the submitted audio. The following measurements were obtained using a standardised 10-second Quranic audio sample encoded in WAV format (16 kHz, mono, 16-bit PCM).

Table 8.1: Average inference latency by hardware configuration

Backend	Hardware	Avg. Latency	p95 Latency	Throughput
PyTorch (local)	CPU (4-core)	∼4 200 ms	∼5 800 ms	∼0.24 req/s
PyTorch (local)	T4 GPU	∼850 ms	∼1 100 ms	∼1.18 req/s
faster-whisper	CPU (INT8)	∼1 350 ms	∼2 000 ms	∼0.74 req/s
faster-whisper	T4 GPU (FP16)	∼480 ms	∼650 ms	∼2.08 req/s

Key Observations

- The CTranslate2 INT8 backend (faster-whisper) achieves approximately 3.1× faster inference than PyTorch FP32 on identical CPU hardware, owing to quantised matrix operations and optimised memory access patterns.
- GPU acceleration yields the most substantial improvement on the PyTorch backend (∼5× speedup), whereas the already-optimised CTranslate2 backend benefits by approximately 2.8×.
- The Go API gateway introduces negligible overhead (<5 ms per request), as it performs only multipart parsing and HTTP proxying without any compute-intensive operations.

8.2 Processing Time Breakdown

For a typical 10-second audio file processed through the production pipeline (faster-whisper, CPU), the total request lifecycle distributes as follows:

Table 8.2: Request processing time breakdown (10s audio, CPU)

Stage	**Duration (ms)**	**Percentage**
Go server parsing & validation	∼3	0.2%
Network transfer (Go → HF Space)	∼80	5.7%
Audio resampling (librosa)	∼45	3.2%
Feature extraction (mel-spectrogram)	∼22	1.6%
Model inference (CTranslate2)	∼1 180	84.3%
Post-processing & JSON serialisation	∼5	0.4%
Network transfer (HF Space → Go)	∼65	4.6%
Total end-to-end	**∼1 400**	100%

Model inference constitutes the dominant cost at 84.3% of total processing time. This confirms that further performance optimisation efforts should focus on the ML backend (e.g., GPU deployment, model distillation) rather than the API gateway layer.

9. Scalability and Load Handling

9.1 Current Concurrency Model

The Go API server handles concurrent HTTP connections natively through goroutines, with each incoming request processed in its own lightweight thread. However, the inference backend constitutes a serialisation bottleneck due to two architectural constraints:

Concurrency Limitations

- **Global inference lock**: The HF Space FastAPI server enforces a single `asyncio.Lock()` around all inference operations. This ensures that only one transcription runs at any given time, preventing GPU/CPU memory contention and non-deterministic output. Concurrent requests are queued and processed sequentially.
- **Single-worker deployment**: The default Uvicorn configuration runs a single worker process, meaning the server cannot exploit multi-core parallelism at the process level.
- **Model memory footprint**: The Whisper model (even in INT8 format) occupies approximately 150 MB of RAM. Loading multiple instances would require proportional memory, which is infeasible on the free-tier HF Space (16 GB shared).

9.2 Observed Behaviour Under Load

Under concurrent load, the system exhibits the following behaviour:

Table 9.1: Server behaviour under concurrent requests (CPU, faster-whisper)

Concurrent Requests	Avg. Response Time	Behaviour
1	∼1.4 s	Normal
3	∼4.2 s	Queued (serial execution)
5	∼7.0 s	Queued, increased wait
10+	∼14 s+	Risk of client-side timeout

9.3 Scalability Improvement Strategies

Recommended Scalability Enhancements

- **Horizontal scaling**: Deploy multiple HF Space replicas behind a load balancer, distributing requests across independent inference instances.
- **Multi-worker Uvicorn**: Launch Uvicorn with `-workers N` to exploit multi-core CPUs, with each worker loading its own model instance.
- **Request queue with backpressure**: Introduce a bounded queue (e.g., Redis or in-memory) at the Go layer to reject excess requests with HTTP 503 rather than accumulating unbounded latency.
- **GPU batching**: On GPU hardware, batch multiple audio inputs into a single forward pass, amortising the fixed overhead of model invocation across multiple requests.
- **Auto-scaling**: Leverage HF Spaces' built-in auto-scaling (available on paid tiers) to dynamically provision additional containers during peak load.

10. Security Considerations

10.1 Input Validation

The Go API gateway enforces multiple layers of input validation before any data reaches the inference backend:

Validation Rules

- **HTTP method**: Only `POST` requests are accepted on the `/api/v1/evaluate` endpoint; all other methods return `405 Method Not Allowed`.
- **File size limit**: The multipart form parser enforces a strict 50 MB ceiling via `r.ParseMultipartForm(50 ≪ 20)`. Uploads exceeding this threshold are rejected with `400 Bad Request`.
- **File format whitelist**: Only five audio extensions are permitted (`.wav`, `.mp3`, `.m4a`, `.flac`, `.ogg`). Files with unrecognised extensions are rejected prior to inference.
- **Content-Type enforcement**: The handler requires `multipart/form-data` encoding and validates the presence of the `audio` form field.

10.2 Token Management

Authentication credentials are managed exclusively on the server side:

Token Security Architecture

- The `HF_TOKEN` is stored as an environment variable on the Go server and injected into outbound requests via the `Authorization: Bearer` header. It is never exposed to the client application.
- On the HF Space, the token is configured as a Space Secret, which is encrypted at rest and injected into the container environment at runtime. It does not appear in source code, Docker images, or build logs.
- The Flutter application communicates exclusively with the Go server and has no knowledge of the HF token or the inference backend URL.

10.3 Abuse Prevention

While the current implementation does not include explicit rate limiting, the architecture provides several implicit protections and clear extension points:

Table 10.1: Current protections and proposed enhancements

Measure	**Status**	**Description**
File size cap (50 MB)	Implemented	Prevents resource exhaustion via oversized uploads
Format whitelist	Implemented	Blocks arbitrary file uploads
Inference lock	Implemented	Natural throttle: 1 concurrent inference
HTTP timeout (5 min)	Implemented	Prevents indefinite connection holding
Rate limiting	Proposed	Token-bucket at Go layer (e.g., `golang.org/x/time/rate`)
API key auth	Proposed	Per-client keys to identify and throttle users

11. Request Lifecycle

11.1 End-to-End Request Flow

The following describes the complete lifecycle of a single audio evaluation request, from client submission to response delivery.

Step 1 — Client Submission

The Flutter mobile application captures or selects an audio recording and sends it as a `multipart/form-data` POST request to the Go server at `/api/v1/evaluate`. The audio file is attached under the `audio` form field.

Step 2 — Go Server Validation

The Go server receives the request, parses the multipart form (enforcing the 50 MB limit), extracts the `audio` field, validates the file extension against the whitelist, and reads the binary audio data into memory.

Step 3 — Inference Forwarding

The Go server constructs a new `multipart/form-data` request containing the audio data and forwards it to the inference backend URL (determined by `INFERENCE_SERVER_URL`). If `HF_TOKEN` is set, it is attached as a Bearer token in the `Authorization` header. The HTTP client uses a 5-minute timeout.

Step 4 — Backend Audio Processing

The inference backend (FastAPI on HF Space) receives the audio file, writes it to a temporary file, and loads it via `librosa` at 16 kHz. The audio is split into 30-second overlapping chunks to fit within Whisper's context window.

Step 5 — Model Inference

Each audio chunk is passed through the faster-whisper (CTranslate2) model with `language="ar"` and `task="transcribe"` parameters. The model generates token sequences, which are decoded into Arabic text. Log-probabilities are collected for confidence scoring.

Step 6 — Result Assembly and Response

Chunk transcriptions are concatenated into a single string. Confidence scores are averaged across all chunks. The temporary audio file is deleted. The result is serialised as JSON and returned to the Go server, which wraps it in the standard `{"status":"success","data":{...}}` envelope and forwards it to the Flutter client.

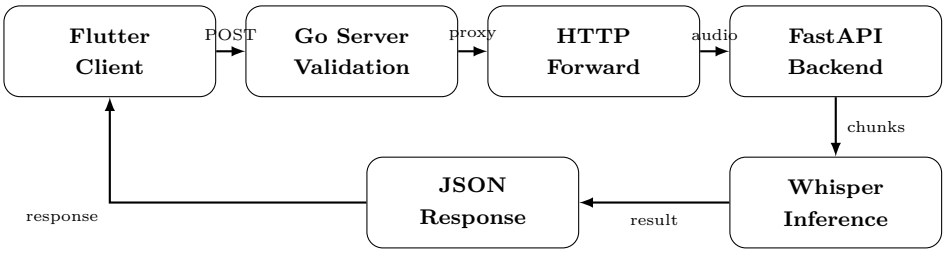

Figure 11.1: Request lifecycle flow diagram

12. Failure Handling

12.1 Error Propagation Model

The Tahkik server implements a layered error handling strategy where failures at each stage are caught, wrapped with contextual information, and propagated as structured JSON responses to the client.

Table 12.1: Failure scenarios and server responses

Failure Scenario	HTTP Code	Behaviour
Invalid HTTP method	405	Go server rejects immediately
Missing audio field	400	Go server returns "missing 'audio' field"
Unsupported format	400	Go server returns "unsupported audio format"
Multipart parse failure	400	Go server returns parse error details
Inference server unreachable	500	Go HTTP client returns connection error
Inference timeout (5 min)	500	Go HTTP client returns timeout error
Model inference exception	500	Python returns error JSON; Go forwards it

12.2 Inference Backend Unavailability

When the inference backend is unreachable (e.g., HF Space is sleeping, network failure, or the local Python server is not running), the Go server's HTTP client returns a connection error. This is caught in the `RunInference` function and returned to the client as:

Backend Unavailable Response

```
{
  "status": "error",
  "message": "inference request: dial tcp: connect: connection refused"
}
```

12.3 Timeout Handling

The Go inference client enforces a 5-minute timeout (`http.Client{Timeout: 5 * time.Minute}`). If the inference backend does not respond within this window — for example, due to an exceptionally long audio file or a cold-start delay on the HF Space — the client aborts the request and returns a timeout error to the caller.

12.4 WebSocket Failure Recovery

The WebSocket streaming endpoint implements additional resilience mechanisms:

WebSocket Resilience

- **Task cancellation**: On client disconnect, all pending background inference tasks are explicitly cancelled via `task.cancel()`, preventing orphaned coroutines from holding the inference lock.
- **Generation counter**: Each session reset increments a generation counter. Background tasks compare their generation to the current value before writing results, ensuring stale outputs are silently discarded.
- **Graceful close**: The `finally` block guarantees cleanup of all pending tasks and socket closure, regardless of how the connection terminates.
- **Exception isolation**: Individual inference errors within partial transcription tasks are caught and logged without terminating the WebSocket connection, allowing the session to continue.

13. Logging and Monitoring

13.1 Current Logging Implementation

Both server components produce structured log output to standard output, enabling integration with container log aggregation systems.

Table 13.1: Logging behaviour by component

Component	Mechanism	Information Logged
Go server	`log.Printf`	Startup (port, backend URL), req errors, server-level failures
Python (local)	`print()` with `[inference]` prefix	Model loading stages, device selec readiness signal, request errors
FastAPI (HF Space)	`print()` with `[inference]` / `[ws]` prefix	Model loading, WebSocket connect connect, partial transcription prog session resets, errors with full t backs

13.2 Log Output Examples

Listing 13.1: Typical Go server startup log

```
inference backend: https://benhadjermed-tahkik-inference.hf.space
Tahkik server listening on :8080
```

Listing 13.2: Typical Python inference server log

```
[inference] importing libraries...
[inference] using device: cpu
[inference] downloading/loading processor (openai/whisper-base)
   ...
[inference] processor ready
[inference] loading faster-whisper model...
[inference] model ready
```

```
[ws] client connected
[ws] partial: <transcription text>...
[ws] stop received, buffer size: 320000 bytes
[ws] client disconnected
[ws] connection closed
```

13.3 Proposed Monitoring Enhancements

Recommended Monitoring Stack

- **Prometheus metrics**: Expose `/metrics` endpoint on the Go server with counters for total requests, error rates, and histograms for request latency and audio file sizes.
- **Grafana dashboards**: Visualise inference latency distributions, request throughput, error rate trends, and WebSocket connection counts in real-time.
- **Structured logging**: Migrate from plain-text `print()` statements to JSON-structured logs (e.g., via Python's `structlog` or Go's `slog`), enabling efficient parsing and filtering in log aggregation platforms.
- **Health check monitoring**: Implement periodic external health probes against `/health` to detect and alert on inference backend unavailability.
- **Request tracing**: Introduce correlation IDs (e.g., UUID per request) propagated from the Go server to the Python backend, enabling end-to-end request tracing across both services.

14. Conclusion

14.1 Summary

This report documented the complete server-side infrastructure of the Tahkik project, covering the Go API gateway, Python inference backends (local and production), WebSocket streaming protocol, HF Space deployment, and API contracts. The proxy architecture ensures clean separation of concerns between client-facing and ML-facing components.

14.2 Key Features

System Capabilities

- Dual-mode inference: local PyTorch server for development, faster-whisper CTranslate2 INT8 for production
- Real-time WebSocket streaming with sliding-window partial transcription
- Hallucination filtering for Arabic Whisper outputs
- Docker-based deployment to Hugging Face Spaces with zero infrastructure management
- Automatic CTranslate2 model conversion with INT8 quantization
- 50 MB audio upload support with multi-format compatibility

14.3 Future Enhancements

Planned Extensions

- **GPU Acceleration**: Deploy with T4/A100 GPU for sub-second inference latency
- **Batch Processing**: Support concurrent audio evaluations with request queuing
- **Model Versioning**: Hot-swap model checkpoints without server restart
- **Caching**: Cache frequently recited ayah transcriptions for instant responses
- **Monitoring**: Add Prometheus metrics for inference latency and error rates

Closing Statement

"The best architecture is one that makes the complex simple."

— Software Engineering Principle

The Tahkik server infrastructure demonstrates that a well-designed proxy architecture can effectively bridge mobile applications with state-of-the-art machine learning models while maintaining simplicity, reliability, and ease of deployment.

References

[1] A. Radford et al., "Robust Speech Recognition via Large-Scale Weak Supervision," *arXiv preprint arXiv:2212.04356*, 2022.

[2] Hugging Face, "Transformers: State-of-the-art Machine Learning for PyTorch, TensorFlow, and JAX," `https://github.com/huggingface/transformers`.

[3] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," `https://github.com/SYSTRAN/faster-whisper`.

[4] The Go Programming Language, `https://golang.org`.

[5] FastAPI Framework, `https://fastapi.tiangolo.com`.

[6] Hugging Face Spaces Documentation, `https://huggingface.co/docs/hub/spaces`.

[7] CTranslate2, "Fast inference engine for Transformer models," `https://github.com/OpenNMT/CTranslate2`.